DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Design and Analysis of Algorithms

ASSIGNMENT 1 – CO1

Algorithm fundamentals, efficiency analysis, asymptotic analysis, recurrence analysis

---

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Case study: University Student Record Retrieval by Registration Number

---

Student Name	**Dinesh R**
Student Reg. No.	**192372209**
Course	Design and Analysis of Algorithms
Assignment	Assignment 1 (CO1) – Total Marks 50
Topic	Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System
Duration	1 week

Tasks 1 – 12: design · basic operation · complexity derivation · best/average/worst case · asymptotics · recurrence · implementation · experiments · theory vs experiment · recommendation · Day-4 design challenge

Student Declaration and Answer Map

I **Dinesh R** (Reg. No. **192372209**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____

Where each task is answered

Table 1. Mapping of the 12 tasks (total 50 marks) to the sections of this report.

Task	Title	Marks	Section
1	Design the Algorithm	5	Task 1 (three strategies, pseudocode, preprocessing)
2	Identify the Basic Operation	2	Task 2
3	Derive the Time-Complexity Function	6	Task 3 (operation counts, preprocessing, total cost)
4	Best, Average and Worst Case	4	Task 4
5	Asymptotic Analysis	3	Task 5
6	Formulate the Recurrence	3	Task 6
7	Solve the Recurrence	4	Task 7
8	Implement and Test	4	Task 8 and Appendix A
9	Record Experimental Performance	3	Task 9
10	Compare Theory with Experiment	4	Task 10
11	Recommend and Defend the Final Design	4	Task 11
12	Day-4 Design Challenge (A, B, C, D)	8	Task 12

Evidence produced for this report

- Working Python programs for all three strategies (`algos.py`), an experiment driver (`exp.py`) and a correctness test suite (`tests.py`); the key code is listed in Appendix A.
- Measured data for n = 1,000, 10,000, 100,000 and 1,000,000 student records; every number in the tables and graphs comes from these runs.
- Mathematical derivations that are compared directly with the measurements in Task 10.

Contents

Student Declaration and Answer Map 2

1. Problem Context, Assumptions and Method 4

Task 1 – Design the Algorithm (5 marks) 6

Task 2 – Identify the Basic Operation (2 marks) 9

Task 3 – Derive the Time-Complexity Function (6 marks) 10

Task 4 – Analyse Best, Average and Worst-Case Behaviour (4 marks) 13

Task 5 – Perform Asymptotic Analysis (3 marks) 16

Task 6 – Formulate the Recurrence Relation (3 marks) 18

Task 7 – Solve the Recurrence Relation (4 marks) 19

Task 8 – Implement and Test the Algorithms (4 marks) 21

Task 9 – Record Experimental Performance (3 marks) 23

Task 10 – Compare Theory with Experimental Behaviour (4 marks) 26

Task 11 – Recommend and Defend the Final Design (4 marks) 29

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt (8 marks) 30

Conclusion 36

Appendix A – Source Code (key parts) 36

Appendix B – Raw Measurements 40

Appendix C – References 40

1. Problem Context, Assumptions and Method

1.1 The problem

A university stores one record per student, identified by a unique registration number. Staff repeatedly ask the same question: *does the record with registration number K exist, and where is it?* The database grows from about 1,000 records to more than 1,000,000 records, so a method that is acceptable for a small department file may become unusable for the whole university. This report designs three meaningfully different search strategies, derives their cost mathematically, implements and measures them, and finally re-evaluates the design for a demanding operating condition: 50,000 search requests per hour on 1,000,000 records, with new records added only once per day.

1.2 Definitions and assumptions

Table 2. Definitions and assumptions.

Item	Definition used in this report
Record and key	A record is identified by a unique 9-digit integer registration number (the key). Keys are unique.
Input size n	The number of records in the database.
Search outcome	**Successful** search: the key is in the database. **Unsuccessful** search: it is not.
Probability p	Probability that a requested key exists. Tasks 3–11 report both outcomes separately; Task 12 uses p = 0.9, because most staff queries concern existing students.
Position of a present key	Every stored key is equally likely to be requested (uniform model), unless stated otherwise.
Initial file order	The raw database file is **unsorted** (records are in arrival order).
Preprocessing	Any work done before the first search (sorting, building a table). It is always included in the efficiency analysis, as the question requires.
Cost model	The number of key comparisons (the basic operation) is the primary measure; measured running time is the secondary measure.
Logarithms	Base 2 unless stated.

1.3 Experimental environment and data

Table 3. Experimental set-up.

Item	Value
Language / interpreter	Python 3.12.3 (CPython)
Platform	Linux-6.18.44-fc-v114-x86_64-with-glibc2.39, x86_64, one logical CPU
Timer	time.perf_counter(); garbage collection disabled during each timed run; median of repeated runs
Dataset sizes	n = 1,000; 10,000; 100,000; 1,000,000
Keys	Unique 9-digit integers drawn at random from 100,000,000 – 999,999,999 with the fixed seed 192372209 + n, stored in random (unsorted) order
Queries	Random present keys and random absent keys, generated from fixed seeds, so every algorithm receives the same queries at each size

Python is an interpreted language, so absolute times are larger than they would be in C, but the *growth* of comparison counts and of running time with n is the same in any language. Where interpreter overhead

affects the comparison between theory and measurement, this is discussed in Task 10.

1.4 Method of the report

1. Task 1 designs the strategies and fixes their preprocessing.
2. Tasks 2 to 7 are analytical: basic operation, exact cost functions, best, average and worst cases, asymptotic notation, and the recurrence of the recursive algorithm.
3. Tasks 8 and 9 implement the algorithms, test their correctness and collect data. Task 10 compares the data with the theory.
4. Tasks 11 and 12 give a recommendation and re-evaluate it under the changed operating conditions.

1.5 Summary of principal findings

Table 4. Headline results at n = 1,000,000 (all measured).

Strategy	Preprocessing	Search at $n = 10^6$ (successful)	Measured time per search	Memory
A Linear search (iterative)	none	496,327 comparisons	23.75 ms	n references
B Binary search (recursive) on sorted file	merge sort: 18,673,175 comparisons, 5.10 s	19.0 comparisons	7.62 µs	n references
C Hash table with chaining	build: n insertions, 917.14 ms	1.50 comparisons	1.03 µs	≈ 7 × the sorted file

Headline conclusion

Binary search and hashing both scale to a million records with negligible search cost, whereas linear search needs about half a million comparisons per successful search. The preprocessing cost is repaid after only a few hundred searches. For the Day-4 workload (50,000 searches per hour, daily updates) the recommended design is the **modified** binary-search design: keep the sorted file, but replace the daily full re-sort by sorting the small daily batch and merging it in linear time.

Task 1 – Design the Algorithm (5 marks)

Three algorithmic strategies are proposed. They differ in their **design technique**, not only in their implementation: brute force, decrease-and-conquer, and space–time trade-off by hashing. The list below also states the preprocessing each one needs, which is part of its cost.

Table 5. The three proposed strategies.

	Strategy A	Strategy B	Strategy C
Name	Sequential (linear) search	Binary search on a sorted file	Hash table with chaining
Design technique	Brute force	Decrease and conquer (halving) after a divide-and-conquer sort	Space–time trade-off (hashing)
Style used	**Iterative**	**Recursive**	Iterative (chain scan)
Preprocessing	None	Sort the keys: merge sort	Build the table: insert all n keys
Extra structure	None	Sorted array	Array of m = 1,000,003 chains (prime m ≥ n)
Keeps order of keys?	No	Yes	No

T1.1 Strategy A – sequential search (brute force, iterative)

Basic idea. Examine the records one after another, starting at the first, and compare each registration number with the key K. Stop when a match is found or when the end of the file is reached. Nothing is assumed about the order of the records.

Preprocessing. None. The database can be searched exactly as it is stored.

```
LinearSearch(A[0..n-1], K)
  FOR i <- 0 TO n - 1 DO
    IF A[i] = K THEN RETURN i        // record found at position i
  END FOR
  RETURN -1                          // not in the database
```

Pseudocode of Strategy A (iterative).

Why it is investigated. It is the baseline against which every other strategy must be justified. It is also the only method that needs no preprocessing and no ordering, and so it is the correct choice for tiny databases or for one-off searches. Its inclusion lets the report measure exactly *how much* the other strategies gain.

T1.2 Strategy B – binary search on a sorted file (decrease and conquer, recursive)

Basic idea. If the file is sorted by registration number, one comparison with the middle record decides which half of the file can contain K, so half of the remaining records can be discarded after every comparison. The problem is reduced by a constant factor (1/2) at each step.

Preprocessing. The file must first be sorted. Merge sort is used because its cost is guaranteed to be $\Theta(n \log n)$ in every case and it is itself a clear divide-and-conquer algorithm whose cost can be derived with a recurrence (Task 7). The sorted copy is kept for all later searches.

```
MergeSort(A[0..n-1])                        Merge(L, R)
  IF n <= 1 THEN RETURN A                     i <- 0 ; j <- 0 ; B <- empty list
  mid <- floor(n / 2)                         WHILE i < |L| AND j < |R| DO
  L <- MergeSort(A[0..mid-1])                   IF L[i] <= R[j] THEN append L[i] to B ; i <- i + 1
  R <- MergeSort(A[mid..n-1])                   ELSE append R[j] to B ; j <- j + 1
  RETURN Merge(L, R)                          append the rest of L and the rest of R to B
                                              RETURN B
```

Pseudocode of the preprocessing step of Strategy B (merge sort).

```
BinarySearch(A[lo..hi], K)                  // initial call: BinarySearch(A, 0, n - 1, K)
  IF lo > hi THEN RETURN -1                   // empty range: K is not present
  mid <- floor((lo + hi) / 2)
  IF A[mid] = K THEN RETURN mid
  IF K < A[mid] THEN RETURN BinarySearch(A, lo, mid - 1, K)
  ELSE RETURN BinarySearch(A, mid + 1, hi, K)
```

Pseudocode of Strategy B (recursive search).

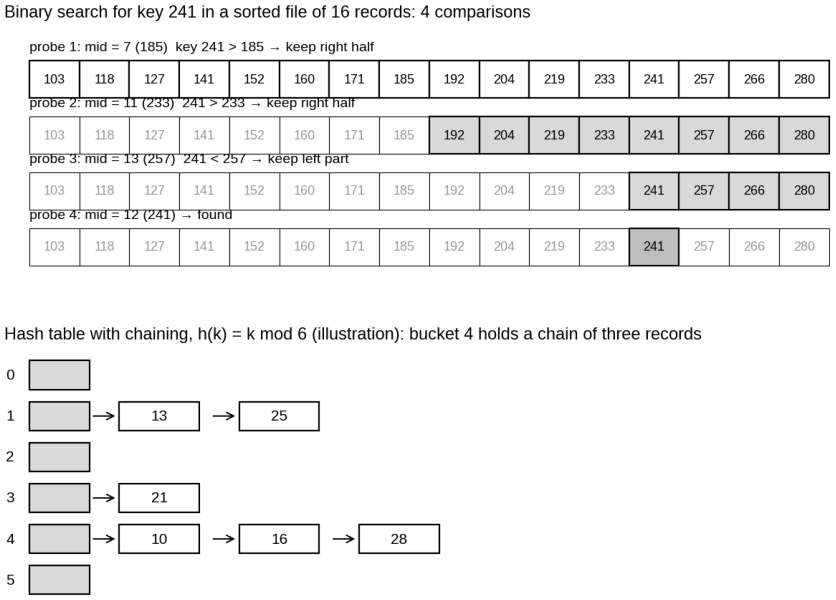


Figure 1. *Top: binary search for key 241 in a sorted file of 16 records finds it in 4 comparisons. Bottom: the layout of a hash table with chaining (Strategy C).*

Why it is investigated. It requires no extra memory beyond the sorted array, gives a guaranteed worst case of $\lfloor \log_2 n \rfloor + 1$ comparisons, and keeps the keys in order, which also supports range queries such as "all registration numbers of one batch". It is the recursive algorithm that is analysed by recurrence in Tasks 6 and 7.

T1.3 Strategy C – hash table with chaining (space–time trade-off)

Basic idea. Instead of finding the record by comparing keys, compute where it should be. A hash function h(K) = K mod m maps each registration number to one of m buckets; keys that land in the same bucket are stored in a short chain. A search computes h(K) and scans only that chain. The table size m is a prime number slightly above n, so the load factor $\alpha = n/m$ is close to 1 and the average chain is about one record long.

Preprocessing. Create m empty chains and insert all n keys. Each insertion appends to one chain, so the build is a single pass over the file.

```
BuildHash(A[0..n-1])                      HashSearch(K)
  m <- smallest prime >= n                  i <- K mod m
  create m empty chains T[0..m-1]           FOR each record x in chain T[i] DO
  FOR each key x in A DO                      IF x = K THEN RETURN found
    append x to chain T[x mod m]            RETURN not found
```

Pseudocode of Strategy C: build (preprocessing) and search.

Why it is investigated. Hashing represents the opposite end of the design space from sorting: it spends memory to obtain an expected constant-time search that does not depend on n at all. It is the natural competitor to binary search for exact-match lookups, so the question of which of the two is better for the university can only be answered by analysing and measuring both.

T1.4 Strategies considered but not selected

Table 6. Alternatives that were considered.

Alternative	Reason for not selecting it as one of the three
Interpolation search	Needs sorted data like Strategy B and is not the same idea as Strategy C; it gains only when keys are uniformly distributed, which cannot be guaranteed for registration numbers that follow year/department codes.
Binary search tree / B-tree	Mainly valuable when records are inserted continuously; here updates occur once per day, so a plain sorted array with batch merging is simpler and faster.
Trie / radix structures	Optimised for variable-length string keys; the registration number is a fixed 9-digit integer.
Iterative and recursive forms of the same algorithm	Not counted as different approaches, as the question states.

T1.5 How the three strategies differ in the cost they accept

The three strategies make different trade-offs between preprocessing, search time and memory. Strategy A pays nothing in advance but pays the maximum at every search. Strategy B pays Θ(n log n) once to make every later search cost Θ(log n). Strategy C pays Θ(n) once and also additional memory to make later searches cost Θ(1) on average. The following tasks quantify these trade-offs.

Result of Task 1

Three distinct strategies have been designed: A (brute force, iterative), B (decrease-and-conquer with merge-sort preprocessing, recursive), and C (hashing, space–time trade-off). Each has pseudocode, a stated preprocessing step and a justification for investigation.

Task 2 – Identify the Basic Operation (2 marks)

The basic operation of an algorithm is the operation that is executed most often and whose count therefore determines the growth of the running time. For a search problem this is a **key comparison**, but the place where it occurs, and the way its count changes with n, are different for each strategy.

Table 7. Basic operation of each strategy.

	Strategy A – linear	Strategy B – binary (and its merge-sort preprocessing)	Strategy C – hash table
Basic operation	Comparison A[i] = K	**Search:** comparison of K with A[mid]. **Preprocessing:** comparison L[i] ≤ R[j] in Merge	Comparison x = K with the records of one chain (the hash computation K mod m is constant work)
Where it occurs	Body of the single FOR loop, once per record examined	Once per recursive call; once per element placed in the output list of Merge	Body of the chain-scan loop, once per record in the chain of bucket h(K)
Why appropriate	Every other step (loop counter increment and test) is executed at most once per comparison, so the comparison count is proportional to total work	Each call performs one comparison and a constant amount of other work; in Merge each comparison moves exactly one element to the output	The chain scan is the only part whose length depends on the data; hash computation and the array access cost the same for every key
How the count changes with n	Grows in proportion to n: more records, more comparisons	Search: grows by one each time n doubles (logarithmically). Sort: grows slightly faster than n	Does not grow with n if the load factor α = n/m is kept constant; depends on α

T2.1 Strategy A

The loop of LinearSearch contains one comparison A[i] = K. The loop test (i < n) and the increment of i execute once per iteration too, but they are not independent of the comparison: the loop body runs exactly as many times as the comparison does. Therefore the count of comparisons is a faithful measure of the running time. If the number of records is multiplied by 10, a search that has to travel through the whole file performs 10 times as many comparisons.

T2.2 Strategy B

In the recursive search the work of one call is: test lo > hi, compute mid, compare K with A[mid], and make at most one recursive call. All of this is constant except that the recursion continues; the number of calls equals the number of comparisons. The calls form a single chain, because after every comparison the range is reduced to at most half of its size. Starting from n records the range becomes n/2, n/4, … and a range of size 1 is reached after about $\log_2 n$ halvings. The count of comparisons therefore changes very slowly with n: a database ten times as large needs only about 3.3 more comparisons ($\log_2 10 \approx 3.32$).

Preprocessing by merge sort has its own basic operation: the comparison L[i] ≤ R[j] inside Merge. Merging two sorted lists of total length k needs between k/2 and k − 1 such comparisons, and merge sort merges at every one of its about $\log_2 n$ levels, so the number of comparisons is proportional to n log n.

T2.3 Strategy C

A hash search does not compare K with a large part of the file. It computes i = K mod m, jumps directly to chain T[i] and compares K with the records in that chain only. The expected length of the chain is the load factor α = n/m. Because m is chosen proportional to n, α stays near 1 as n grows, and so the number of comparisons stays near 1 to 1.5 whatever the size of the file. The count therefore depends on how evenly the hash function spreads the keys, not on n.

Result of Task 2

Basic operations: A – comparison A[i] = K in the loop; B – comparison with A[mid] per call (search) and L[i] ≤ R[j] in Merge (preprocessing); C – comparison with the records of one chain. Counts grow as n, log n and (essentially) a constant respectively.

Task 3 – Derive the Time-Complexity Function (6 marks)

For every strategy the input size is n, the number of records. The running time is expressed as T(n) = c·C(n) + d, where C(n) is the number of basic operations, c is the cost of one operation and d is the fixed overhead; the order of growth is that of C(n). The derivation therefore concentrates on C(n) and on the preprocessing cost P(n). For q searches the **overall cost** is

$$\text{Total}(n, q) = P(n) + q \cdot C(n) \tag{1}$$

T3.1 Strategy A – sequential search

Operation count

The comparison A[i] = K is made once for each position inspected. If K is at position i (counting from 1), the loop makes exactly i comparisons. If K is absent the loop inspects all n positions and makes n comparisons.

Average number of comparisons

With probability p the key is present and equally likely to be at any of the n positions, so position i has probability p/n; with probability 1 − p it is absent.

$$C_A(n, p) = \Sigma_{i=1}^{n}\, i\cdot(p/n) + n(1 - p) \qquad \textit{expected value over all outcomes}$$

$$= (p/n)\cdot n(n + 1)/2 + n(1 - p) \qquad \textit{sum of the first n integers}$$

$$= p(n + 1)/2 + n(1 - p) \qquad \textit{simplify}$$

$$= n(1 - p/2) + p/2 \qquad \textit{collect terms}$$

For a successful search only (p = 1) this is (n + 1)/2, and for an unsuccessful search (p = 0) it is n. The worst case is n comparisons.

Preprocessing and total cost

$$P_A(n) = 0,\ \text{Total}_A(n, q) = q\cdot(n + 1)/2 \text{ (successful)},\ q\cdot n \text{ (unsuccessful)} \tag{2}$$

Table 8. Operation counts of Strategy A (derived).

n	(n + 1)/2 (successful average)	n (unsuccessful / worst)
1,000	500.5	1,000
10,000	5,000.5	10,000
100,000	50,000.5	100,000
1,000,000	500,000.5	1,000,000

T3.2 Strategy B – sorting, then binary search

Operation count of the search

Let $C_B(n)$ be the number of comparisons in the worst case. Each comparison reduces the range from n to at most $\lfloor n/2 \rfloor$, and the search stops when the range is empty. The solution of $C_B(n) = C_B(\lfloor n/2 \rfloor) + 1$ (derived in Tasks 6 and 7) is

$$C_B(n) = \lfloor \log_2 n \rfloor + 1 \tag{3}$$

Average case. To count exactly, take $n = 2^k - 1$ so that the search tree is perfectly balanced. Level j of the tree holds 2^{j-1} keys, each found after j comparisons:

$C_{succ} = (1/n)\cdot\Sigma_{j=1}^{k} j\cdot 2^{j-1}$	*j comparisons for each of the 2^{j-1} keys at level j*
$= ((k-1)\cdot 2^k + 1)/(2^k - 1)$	*standard sum $\Sigma\, j\cdot 2^{j-1} = (k-1)2^k + 1$*
$= (k-1) + k/(2^k - 1)$	*write $(k-1)2^k + 1 = (k-1)(2^k - 1) + k$*
$\approx \log_2(n+1) - 1$	*since $k = \log_2(n+1)$ and $k/(2^k - 1)$ is tiny*

An unsuccessful search ends after about $k = \log_2(n + 1)$ comparisons, because every one of the n + 1 gaps between keys lies at the bottom of the balanced tree.

Preprocessing: cost of merge sort

Merge sort splits the file into halves, sorts each recursively and merges. Merging two sorted lists of lengths a and b needs at most a + b − 1 comparisons. For $n = 2^k$ the worst-case count W(n) satisfies $W(n) = 2W(n/2) + n - 1$ with $W(1) = 0$. Unrolling the recurrence i times,

$W(n) = 2^i\cdot W(n/2^i) + \Sigma_{j=0}^{i-1} 2^j(n/2^j - 1)$	*each level j contributes 2^j merges of size n/2^j*
$= 2^i\cdot W(n/2^i) + i\cdot n - (2^i - 1)$	*each level contributes n; $\Sigma\, 2^j = 2^i - 1$*
$= n\cdot\log_2 n - (n-1)$	*stop at $i = \log_2 n$, where $W(1) = 0$*

$$P_B(n) = W(n) = n\cdot\log_2 n - n + 1 \text{ (worst case)};\ \tfrac{1}{2}\cdot n\cdot\log_2 n \le P_B(n) \le n\cdot\log_2 n - n + 1 \tag{4}$$

For random input the expected number of comparisons is about $n\cdot\log_2 n - 1.25n$, which lies inside these bounds and is compared with the measurement in Task 10. A merge sort also copies each element once per level, so its running time is proportional to the same quantity: $\Theta(n \log n)$.

Overall cost

$$\text{Total}_B(n, q) = n\cdot\log_2 n - n + 1 + q\cdot(\lfloor \log_2 n \rfloor + 1) \tag{5}$$

Table 9. Derived cost of Strategy B.

n	$P_B(n)$ bound n log n − n + 1	expected ≈ n log n − 1.25n	$C_B(n)$ worst	average ≈ $\log_2(n+1) - 1$
1,000	8,967	8,716	10	8.97
10,000	122,878	120,377	14	12.29
100,000	1,560,965	1,535,964	17	15.61
1,000,000	18,931,570	18,681,569	20	18.93

T3.3 Strategy C – hash table with chaining

Model and operation count

Assume simple uniform hashing: each key is equally likely to be placed in any of the m buckets, independently of the others. The table has load factor α = n/m. The length of the chain to which a given key hashes has expected value α (n keys spread over m buckets).

Unsuccessful search. The whole chain is scanned, so the expected number of comparisons is α. **Successful search.** Suppose the sought key was the i-th key inserted. It has i − 1 earlier keys, each of which lies in the same chain with probability 1/m, so the expected number of comparisons for that key is 1 + (i − 1)/m. Averaging over i = 1, …, n:

$$C_C^{\text{succ}} = (1/n)\cdot\Sigma_{i=1}^{n}(1 + (i-1)/m) \qquad \textit{expected comparisons for the i-th inserted key}$$

$$= 1 + (1/(nm))\cdot n(n-1)/2 \qquad \textit{sum of 0, 1, …, n − 1}$$

$$= 1 + (n-1)/(2m) \qquad \textit{simplify}$$

$$= 1 + \alpha/2 - \alpha/(2n) \qquad \textit{with } \alpha = n/m$$

Preprocessing and total cost

The build creates m empty chains (Θ(m) work) and performs n insertions at the end of a chain (Θ(1) each). With m ≈ n:

$$P_C(n) = \Theta(m+n) = \Theta(n)\ \text{Total}_C(n, q) = \Theta(n) + q\cdot(1 + \alpha/2) \tag{6}$$

Worst case. If all keys hash to the same bucket (an adversarial or badly structured key set) the chain contains n keys and a search needs n comparisons, exactly like Strategy A. The worst-case behaviour of hashing is therefore Θ(n); it is avoided in practice by a good prime modulus or by universal hashing (Tasks 4 and 10).

Table 10. Derived expected comparisons of Strategy C.

n	m (prime)	α	successful 1 + α/2	unsuccessful α
1,000	1,009	0.99108	1.4950	0.9911
10,000	10,007	0.99930	1.4996	0.9993
100,000	100,003	0.99997	1.5000	1.0000
1,000,000	1,000,003	1.00000	1.5000	1.0000

T3.4 Combined view: when does preprocessing pay off?

The sorted file (B) is better than Strategy A as soon as $P_B(n) + q\cdot C_B < q\cdot C_A$, that is, when

$$q > P_B(n) / (C_A(n) - C_B(n)) \approx (n \cdot \log_2 n - 1.25n) / ((n + 1)/2) \approx 2(\log_2 n - 1.25) \quad (7)$$

For n = 1,000,000 this gives q > 37.4 comparisons-based searches: after roughly 37 successful searches the sorted file has already paid for its sort. For the hash table the corresponding threshold is q > n/((n + 1)/2 − 1.5) ≈ 2 searches. The comparison-count thresholds are tiny because one sorting pass costs only about $2 \cdot \log_2$ n sequential searches. Task 10 repeats the calculation with *measured times*, which include the interpreter overhead.

Result of Task 3

A: C(n) = n(1 − p/2) + p/2 average, n worst, no preprocessing. B: C(n) = $\lfloor \log_2 n \rfloor + 1$ worst, ≈ $\log_2(n + 1)$ − 1 average, with preprocessing n $\log_2$ n − n + 1 comparisons. C: expected 1 + α/2 (successful) and α (unsuccessful) comparisons, preprocessing Θ(n), worst case n.

Task 4 – Analyse Best, Average and Worst-Case Behaviour (4 marks)

A case is defined by the *input condition* that makes an algorithm do the least, a typical amount, or the most work, for the same n. For search algorithms the conditions concern where the key is (or whether it is present) and, for hashing, how the keys are spread over the buckets. Each case below is explained and then confirmed with measurements (the table at the end of this task).

T4.1 Strategy A – sequential search

Table 11. Cases of Strategy A.

Case	Input condition	Comparisons	Why
Best	K is the **first** record of the file	1	The loop returns immediately after one comparison, whatever n is.
Average	K present with probability p, each position equally likely	n(1 − p/2) + p/2	On average the loop passes through half the file when K is present, and the whole file when K is absent.
Worst	K is the **last** record, or K is **not in** the file	n	The loop cannot stop early: it must inspect every record before it can answer.

Why the difference arises. Linear search uses no information about the data except equality, so a comparison that fails removes only one candidate. The number of comparisons is determined by the position of K alone. The best case is a matter of luck; but the worst case is not an exotic event: every unsuccessful search, which is expected for any mistyped or non-existent registration number, is a worst-case search.

T4.2 Strategy B – binary search

Table 12. Cases of Strategy B.

Case	Input condition	Comparisons	Why
Best	K equals the **middle** record, A[⌊(n − 1)/2⌋], of the whole sorted file	1	The first probe is the middle element, so the very first comparison succeeds.
Average	K is a randomly chosen present key (or a random absent value)	$\approx \log_2(n+1) - 1$ successful; $\approx \log_2(n+1)$ unsuccessful	Level j of the search tree holds 2^{j-1} keys, so half of all keys lie on the last level and the average is close to the worst case.
Worst	K is **absent**, or lies at the deepest level of the search tree	$\lfloor \log_2 n \rfloor + 1$	The range halves after every comparison and the search stops only when the range is empty or has one element left.

Why the difference arises. The best case needs a lucky first probe; only one of the n keys has this property, so its probability is 1/n. In contrast, the number of keys that need exactly j comparisons doubles with every level (1, 2, 4, 8, …), so most keys need nearly the maximum number of comparisons. This is why binary search has an unusually small gap between its average and worst cases: for n = 1,000,000 the worst case is 20 comparisons and the average for a successful search is about 19. The sorted order is the information that converts a failed comparison into the removal of half of the candidates, and it is precisely this that the preprocessing buys.

T4.3 Strategy C – hash table with chaining

Table 13. Cases of Strategy C.

Case	Input condition	Comparisons	Why
Best	K is the first (or only) record of its chain; for an absent key, its bucket is **empty**	1 (present); 0 (absent, empty bucket)	The hash function sends K to a chain of length ≤ 1; the empty-bucket case needs no key comparison at all.
Average	Keys spread evenly over the buckets (simple uniform hashing)	$1 + \alpha/2$ successful; α unsuccessful	Chain lengths are random with mean α; a present key sees on average half of the other keys of its chain.
Worst	**All keys** hash to the same bucket (poor hash function, or structured keys that are multiples of m)	n	The table degenerates into one chain of length n, which is scanned like Strategy A.

Why the difference arises. The cost of a hash search depends on *the distribution of the keys over the buckets*, not on n. With random keys, the fraction of empty buckets is $e^{-\alpha} \approx 36.8\ \%$ for $\alpha \approx 1$, which explains why about one in three unsuccessful searches costs nothing; the measured fractions of empty buckets are 35.5 %, 36.5 %, 36.7 %, 36.8 % for $n = 10^3$, 10^4, 10^5 and 10^6. If, however, an adversary (or a poor choice of m) places all keys in one bucket, hashing loses its entire advantage. The experiment below creates exactly this worst case.

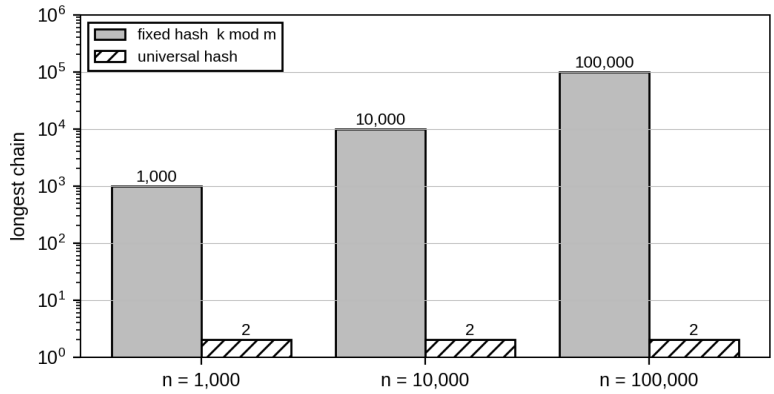


Figure 2. *Longest chain when the n keys are all multiples of the table size m (worst case for the division hash) and when a universal hash function with random parameters is used instead.*

With the fixed hash K mod m, the keys 1·m, 2·m, …, n·m all fall in bucket 0, so the longest chain equals n (1,000; 10,000; 100,000 in the experiment) and a successful search needs on average 503, 5,014 and 47,210 comparisons, which is about n/2 as for linear search. With the universal hash function h(K) = ((aK + b) mod P) mod m, P = $2^{61} - 1$ and random a, b, the same keys produce a longest chain of only 2, 2 and 2: the worst case has been removed by randomisation.

T4.4 Measured evidence for the three cases

Table 14. Measured comparisons: best, average (successful) and worst case. Linear worst case is exact (n); binary and hash worst cases are the maximum seen over 40,000 queries.

n	A best	A avg (succ.)	A worst	B best	B avg (succ.)	B worst	C best	C avg (succ.)	C worst (random keys)
1,000	1	499	1,000	1	8.99	10	1	1.45	5
10,000	1	5,023	10,000	1	12.38	14	1	1.50	6
100,000	1	49,849	100,000	1	15.70	17	1	1.49	8
1,000,000	1	496,327	1,000,000	1	18.96	20	1	1.50	7

Result of Task 4

The best case is a single comparison for all three strategies, but only A and B have a worst case that is certain (n and $\lfloor \log_2 n \rfloor + 1$). The worst case of A coincides with every unsuccessful search; for B the average is within one comparison of the worst case; for C the worst case n occurs only for degenerate key distributions, and it was reproduced and then removed by universal hashing.

Task 5 – Perform Asymptotic Analysis (3 marks)

Asymptotic notation compares the growth of the cost functions of Task 3 when n becomes very large. Recall that O gives an upper bound, Ω a lower bound and Θ a tight bound. Because the cost of a search depends on the input condition, each notation is attached to a particular case.

T5.1 Asymptotic form of each cost function

Table 15. Asymptotic notation for every cost function.

Quantity	Exact / expected count	Asymptotic form	Justification of constants
A: search, worst case	n	Θ(n)	$c_1 = c_2 = 1$, $n_0 = 1$
A: search, best case	1	Θ(1)	constant
A: search, average (p fixed)	n(1 − p/2) + p/2	Θ(n)	for 0 < p ≤ 1 and n ≥ 1: n/2 ≤ C ≤ n
B: search, worst case	⌊log n⌋ + 1	Θ(log n)	for n ≥ 2: log n ≤ ⌊log n⌋ + 1 ≤ 2 log n
B: preprocessing (merge sort)	n log n − n + 1	Θ(n log n)	for n ≥ 4: ½ n log n ≤ n log n − n + 1 ≤ n log n
B: update by full re-sort	n log n − n + 1	Θ(n log n)	as above
C: search, expected	1 + α/2 (α ≤ 1)	Θ(1)	1 ≤ C ≤ 3/2
C: search, worst case	n	Θ(n)	all keys in one chain
C: preprocessing (build)	m + n	Θ(n)	m is a prime in [n, 2n)

Proof for merge sort. For $n \geq 4$ we have $\log_2 n \geq 2$, so $n - 1 \leq \tfrac{1}{2} n \log_2 n$, which gives $n \log_2 n - n + 1 \geq n \log_2 n - \tfrac{1}{2} n \log_2 n = \tfrac{1}{2} n \log_2 n$. The upper bound $n \log_2 n - n + 1 \leq n \log_2 n$ is obvious for $n \geq 1$. Hence $c_1 = 1/2$, $c_2 = 1$, $n_0 = 4$.

Proof for the binary-search count. For $n \geq 2$, $\log_2 n \leq \lfloor \log_2 n \rfloor + 1$ because the floor loses less than 1, and $\lfloor \log_2 n \rfloor + 1 \leq \log_2 n + 1 \leq 2 \log_2 n$. Hence $c_1 = 1$, $c_2 = 2$, $n_0 = 2$.

Remark on the choice of notation. Linear search has no single Θ bound that covers every input: its running time is Ω(1) and O(n), but Θ(1) in the best case and Θ(n) in the worst and average case. Statements such as “linear search is Θ(n)” are therefore always attached to a case.

T5.2 Growth behaviour for very large n

The functions are ranked by their limits. As $n \to \infty$: $\log n / n \to 0$, so $\log n = o(n)$; $n / (n \log n) \to 0$, so $n = o(n \log n)$; and $1/\log n \to 0$, so $1 = o(\log n)$. The ranking of the proposed approaches is therefore

$$\Theta(1) \quad \Theta(\log n) \quad \Theta(n) \text{ (search cost of } C, B, A) \tag{8}$$

$$\Theta(1) \quad \Theta(n) \quad \Theta(n \log n) \text{ (preprocessing cost of } A, C, B) \tag{9}$$

T5.3 What the growth means for 1,000 → 1,000,000 records

Table 16. Growth of cost when the database grows a thousand-fold (theory from Task 3, measurements from Task 9).

Quantity	n = 1,000	n = 1,000,000	Growth factor (theory)	Growth factor (measured)
A: comparisons, successful	498.6	496,327	× 1,000	× 995
B: comparisons, successful	8.99	18.96	× 2.11	× 2.11
C: comparisons, successful	1.45	1.50	× 1.00	× 1.03
B: sort comparisons	8,677	18,673,175	× 2,000	× 2,152
C: build time	445.79 µs	917.14 ms	× 1,000	× 2,057
A: time per successful search	17.50 µs	23.75 ms	× 1,000	× 1,357
B: time per successful search	1.85 µs	7.62 µs	× 2.0 (comparisons)	× 4.1
C: time per successful search	0.16 µs	1.03 µs	× 1.0	× 6.5

Interpretation for the university. A thousand-fold growth of the database multiplies the search cost of linear search by a thousand: a lookup that takes about 18 µs for a department file takes about 24 ms (successful) to 46 ms (unsuccessful) for the whole university. Binary search needs twice as many comparisons (10 instead of 20 in the worst case), and hashing needs the same number of comparisons. Binary search and hashing therefore remain practical for any realistic growth of the database, whereas linear search degrades in direct proportion to the number of students. The price is the preprocessing: it grows by a factor of about 2,000 (n log n) for sorting and by a factor of about 1,000 or slightly more for building the hash table. Section Task 10 discusses why the measured factors for time exceed the theoretical ones for some entries (cache effects).

Result of Task 5

A is $\Theta(n)$ (worst and average) and $\Theta(1)$ (best); B is $\Theta(\log n)$ per search with $\Theta(n \log n)$ preprocessing; C is $\Theta(1)$ expected per search ($\Theta(n)$ worst) with $\Theta(n)$ preprocessing. Growth from 10^3 to 10^6 records: ×1,000 for A, ×2 for B, ×1 for C.

Task 6 – Formulate the Recurrence Relation (3 marks)

The recursive approach of this report is the BinarySearch procedure of Strategy B (Task 1). Its recurrence is derived below directly from the code, call by call, and not quoted from a table of known recurrences. The preprocessing step, merge sort, is also recursive, so its recurrence is derived in the same way as a second example.

T6.1 Recurrence for the recursive search

```
BinarySearch(A, lo, hi, K)
  IF lo > hi THEN RETURN -1                  // (1) base case: empty range
  mid <- floor((lo + hi) / 2)                // (2) constant work
  IF A[mid] = K THEN RETURN mid              // (3) the basic operation: one comparison
  IF K < A[mid] THEN RETURN BinarySearch(A, lo, mid - 1, K)   // (4a) left half
  ELSE RETURN BinarySearch(A, mid + 1, hi, K)                 // (4b) right half
```

The procedure analysed. The numbers refer to the table below.

Table 17. Components of the recurrence for BinarySearch.

Component	Identification from the algorithm
Input size of the original problem	n = hi − lo + 1, the number of records in the range being searched
Size of the recursive subproblem	The left call searches mid − lo = ⌊(n − 1)/2⌋ records; the right call searches hi − mid = ⌊n/2⌋ records. In both cases at most ⌊n/2⌋ records remain. The worst case, which fixes the running time, always follows the larger part, ⌊n/2⌋.
Number of recursive calls	**One**. Statements (4a) and (4b) are alternatives: exactly one of them is executed, so the number of subproblems is a = 1 and the subproblem size is n/b with b = 2.
Work outside the recursive call	Statements (1), (2), (3): one test, one addition and division, and **one key comparison**. This is a constant, f(n) = 1 comparison.
Base case	Empty range, n = 0: the procedure returns at (1) without any key comparison, T(0) = 0. For n = 1 the call performs one comparison and, if K is absent, makes one further call on an empty range, so T(1) = 1 + T(0) = 1.

Assembling the recurrence

In the worst case (K absent, or K found only after the last halving) the call on n records performs its one comparison and then calls itself on at most ⌊n/2⌋ records. Counting key comparisons:

$$T(n) = T(\lfloor n/2 \rfloor) + 1 \text{ for } n \geq 1,\ T(0) = 0 \tag{10}$$

It follows that T(1) = T(0) + 1 = 1, T(2) = T(1) + 1 = 2, T(3) = 2, T(4) = T(2) + 1 = 3. For powers of two, $n = 2^k$, the floor disappears and the recurrence takes the simpler form used in Task 7:

$$T(n) = T(n/2) + 1,\ T(1) = 1 \tag{11}$$

Cross-check against the program. The instrumented program (Task 8) counts one comparison per call. For n = 1,000 the largest count over 40,000 queries is 10, for n = 10,000 it is 14, for n = 100,000 it is 17 and for n = 1,000,000 it is 20. Evaluating recurrence (10) by hand gives T(1,000) = 10, T(10,000) = 14, T(100,000) = 17, T(1,000,000) = 20 (for example 1,000,000 → 500,000 → 250,000 → … → 1, which is 20 values), so the recurrence describes the program exactly.

T6.2 Recurrence for the recursive preprocessing (merge sort)

Table 18. Components of the recurrence for MergeSort.

Component	Identification
Input size	n, the number of records to be sorted
Subproblems	Two: the left half of size ⌈n/2⌉ and the right half of size ⌊n/2⌋
Work outside the recursive calls	Merge of the two sorted halves: at most n − 1 key comparisons (and n element moves)
Base case	n = 1: a single record is sorted, no comparison, T(1) = 0

$$W(n) = W(\lceil n/2 \rceil) + W(\lfloor n/2 \rfloor) + n - 1 \text{ for } n \geq 2,\ W(1) = 0 \tag{12}$$

For $n = 2^k$ this becomes $W(n) = 2W(n/2) + n - 1$ with $W(1) = 0$. Unlike the search, the merge sort makes *two* recursive calls on half-size problems and its non-recursive work is linear, which is why its cost will grow faster than that of the search.

Result of Task 6

Recursive search: a = 1 subproblem of size ≤ n/2, constant work 1, base case T(0) = 0 (T(1) = 1) giving $T(n) = T(\lfloor n/2 \rfloor) + 1$. Recursive sort: $W(n) = W(\lceil n/2 \rceil) + W(\lfloor n/2 \rfloor) + n - 1$, W(1) = 0.

Task 7 – Solve the Recurrence Relation (4 marks)

T7.1 Choice of method

The recurrence $T(n) = T(n/2) + 1$ is of the form $T(n) = aT(n/b) + f(n)$ with a = 1, b = 2, f(n) = 1, so the Master Theorem applies directly. However, the Master Theorem gives only the order of growth. To obtain the **exact** number of comparisons, so that it can be compared with the experiment, the **iteration (backward substitution) method** is used as the main method: the right-hand side is expanded repeatedly until the base case is reached. The result is then confirmed by induction and by the Master Theorem.

T7.2 Solution by iteration (backward substitution)

Take $n = 2^k$ so that n/2 is always an integer.

$T(n) = T(n/2) + 1$ *original recurrence*

$= [\ T(n/4) + 1\ ] + 1 = T(n/4) + 2$ *substitute T(n/2) = T(n/4) + 1*

$= [\ T(n/8) + 1\ ] + 2 = T(n/8) + 3$ *substitute T(n/4) = T(n/8) + 1*

$= [\ T(n/16) + 1\ ] + 3 = T(n/16) + 4$ *substitute T(n/8) = T(n/16) + 1*

…

$= T(n/2^i) + i$ *general pattern after i expansions*

The pattern holds for every i (proof by induction on i): it is true for i = 1; if $T(n) = T(n/2^i) + i$, then replacing $T(n/2^i)$ by $T(n/2^{i+1}) + 1$ gives $T(n) = T(n/2^{i+1}) + (i + 1)$.

Reaching the base case. The expansion stops when $n/2^i = 1$, that is when $i = k = \log_2 n$:

$T(n) = T(n/2^k) + k$ *pattern with i = k*

$= T(1) + \log_2 n$ *since n/2^k = 1*

$= 1 + \log_2 n$ *base case T(1) = 1*

General n. For arbitrary n, let $k = \lfloor \log_2 n \rfloor$, i.e. $2^k \le n < 2^{k+1}$. Then $2^{k-1} \le \lfloor n/2 \rfloor < 2^k$, so $\lfloor n/2 \rfloor$ has floor-logarithm k − 1. If $T(m) = \lfloor \log_2 m \rfloor + 1$ holds for all smaller m, then $T(n) = T(\lfloor n/2 \rfloor) + 1 = (k - 1) + 1 + 1 = k + 1 = \lfloor \log_2 n \rfloor + 1$.

Result

Exact solution: $T(n) = \lfloor \log_2 n \rfloor + 1$ for every $n \ge 1$ (and $T(n) = \log_2 n + 1$ when n is a power of two). Hence $T(n) \in \Theta(\log n)$.

T7.3 Verification of the solution

By induction (guess and verify)

Base: $T(1) = 1 = \log_2 1 + 1$. **Step:** assume $T(n/2) = \log_2(n/2) + 1 = \log_2 n$. Then $T(n) = T(n/2) + 1 = \log_2 n + 1$, which is the claimed form.

By the Master Theorem

With a = 1, b = 2 and f(n) = 1 the watershed function is $n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^0)$, Case 2 applies and $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$, in agreement with the exact solution. Cases 1 and 3 do not apply because f(n) is neither polynomially smaller nor polynomially larger than 1.

By direct evaluation

Table 19. The recurrence, its closed form and the measurement agree.

n	1	2	3	4	8	1,000	10,000	100,000	1,000,000
T(n), recurrence (10)	1	2	2	3	4	10	14	17	20
$\lfloor \log_2 n \rfloor + 1$	1	2	2	3	4	10	14	17	20
measured maximum (Task 9)	–	–	–	–	–	10	14	17	20

T7.4 Solving the recurrence of the preprocessing (merge sort)

For $W(n) = 2W(n/2) + n - 1$, $W(1) = 0$ we have a = 2, b = 2, $f(n) = n - 1 = \Theta(n)$ and watershed function $n^{\log_2 2} = n$. Since $f(n) = \Theta(n)$, Case 2 gives $W(n) = \Theta(n \log n)$. The exact solution was obtained by iteration in Task 3, $W(n) = n \log_2 n - n + 1$. As a check, $W(2) = 2 \cdot 0 + 1 = 1$ and $2 \cdot 1 - 2 + 1 = 1$; $W(4) = 2 \cdot 1 + 3 = 5$ and $4 \cdot 2 - 4 + 1 = 5$; $W(8) = 2 \cdot 5 + 7 = 17$ and $8 \cdot 3 - 8 + 1 = 17$.

T7.5 Interpretation

- **Behaviour of the recursion.** The recursion tree of BinarySearch is a single chain of $\lfloor \log_2 n \rfloor + 1$ calls: one comparison is done at each level and the size halves, so the work is a sum of log n ones. The depth of the recursion equals the number of comparisons, so for n = 1,000,000 at most 20 recursive calls are active at the same time.
- **As n increases.** Doubling the number of records adds exactly one comparison. Growing the database from 10^3 to 10^6 records adds only 10 comparisons (10 → 20). A million-record database is searched with about the same effort as a thousand-record database multiplied by two.
- **Consequence for the design.** The search itself is so cheap that the cost of the system is dominated by the preprocessing: the sort needs $\Theta(n \log n)$ comparisons, which is more than 18 million for $n = 10^6$. This is the cost that is amortised over the searches in Tasks 10 to 12.

- **Memory.** The recursive version uses stack space proportional to the recursion depth, $\Theta(\log n)$ (20 frames at $n = 10^6$); an iterative version would use $\Theta(1)$ but performs the same comparisons.

Result of Task 7

$T(n) = T(\lfloor n/2 \rfloor) + 1$, $T(0) = 0$ has the exact solution $T(n) = \lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$ (iteration, induction and Master Theorem Case 2 agree); the merge-sort recurrence $W(n) = 2W(n/2) + n - 1$ gives $n \log_2 n - n + 1 = \Theta(n \log n)$.

Task 8 – Implement and Test the Algorithms (4 marks)

T8.1 Implementation

All three strategies, and the merge-sort preprocessing, were implemented in Python 3 exactly as in the pseudocode of Task 1. The source code is in the files `algos.py` (algorithms), `tests.py` (correctness tests), `exp.py` (experiment driver) and `figs2.py` (graphs); the central parts are listed in Appendix A.

Table 20. Mapping of algorithms to code.

Strategy	Function	Form	Preprocessing function
A Linear search	linear_search(a, key)	iterative FOR loop	none
B Binary search	binary_search(a, lo, hi, key)	**recursive**, exactly the pseudocode of Task 1	merge_sort(a) (recursive) with merge step
C Hash table	HashIndex.search(key)	iterative scan of one chain	HashIndex.build(keys), m = next_prime(n)

Correspondence with the analysis. Each algorithm exists in two variants that perform the same steps: a *timing variant* without counters, used for all time measurements so that instrumentation does not distort them, and a *counted variant* (`linear_search_counted`, `binary_search_counted`, `merge_sort_counted`, `HashIndex.search_counted`) that increments a counter at exactly the basic operation identified in Task 2. The experiment driver asserts that both variants return identical results.

T8.2 Datasets and test procedure

1. **Generate the data.** For each n in {1,000; 10,000; 100,000; 1,000,000} draw n distinct 9-digit registration numbers with `random.Random(192372209 + n).sample(...)`. The list is in random order and is the raw database. All three algorithms use this same list.
2. **Preprocess.** Sort a copy with merge_sort (Strategy B) and build the hash table (Strategy C). Time both and count the comparisons of the sort.
3. **Generate the queries.** Draw 20,000 random *present* keys and 20,000 random *absent* keys (absent keys are drawn at random and rejected if they are in the database). The same queries are used for B and C; Strategy A, whose search is slow, uses the first 2,000 / 500 / 60 / 20 queries of each kind at $n = 10^3 / 10^4 / 10^5 / 10^6$.
4. **Verify correctness** of every answer against an independent oracle (membership in a Python set), see T8.3.
5. **Count comparisons** with the counted variants (Strategy A: exact count = position + 1 for present keys, verified by actual counted scans on a sample, and n for absent keys).

6. **Measure time** with the timing variants: each batch of queries is timed as a whole with time.perf_counter(), garbage collection disabled, and divided by the number of queries; the median of 3 repetitions is reported (Strategy A at large n: one repetition, because one repetition already contains many scans). Preprocessing is the median of 3 runs (2 at $n = 10^6$).
7. **Record** the results (results.json) and generate tables and graphs from this file only.

T8.3 Correctness tests

A test passes only if the answers of all three algorithms are identical and equal to the oracle (key present or absent), and, for present keys, if the returned position really contains the key.

Table 21. Correctness test cases. (At n = 1,000,000 Strategy A is checked on 20 of the queries because every linear search scans up to a million records.)

Test	Records n	Queries	Result
T1 empty database	0	3	PASS
T2 single record, key present	1	1	PASS
T3 single record, key absent	1	2	PASS
T4 two records, both keys + absent	2	3	PASS
T5 first and last of unsorted file	1,000	2	PASS
T6 smallest and largest key	1,000	2	PASS
T7 middle element (binary best case)	1,000	1	PASS
T8 below minimum / above maximum	1,000	4	PASS
T9 1,000 random present keys	1,000	1,000	PASS
T10 1,000 random absent keys	1,000	1,000	PASS
T11 random mix, n = 1,000	1,000	2,000	PASS
T11 random mix, n = 10,000	10,000	2,000	PASS
T11 random mix, n = 100,000	100,000	2,000	PASS
T11 random mix, n = 1,000,000	1,000,000	2,000	PASS

All 14 test cases pass. They cover the boundary conditions that most often break search code: an **empty** database, a **single** record (present and absent), **two** records, the **first** and **last** records of the unsorted file, the **smallest and largest** keys, the **middle** key (best case of binary search), keys **below the minimum and above the maximum**, and large random mixes of present and absent keys at all four sizes.

T8.4 Consistency of the testing conditions

- Same data: one list per n, shared by all algorithms; seeds are fixed, so a rerun gives the same data and queries.
- Same machine, same interpreter, same timer; garbage collection disabled while timing; no other workload deliberately run.
- Same query set for B and C; Strategy A uses a prefix of that set (random, so unbiased).
- Equivalent outputs: every algorithm answers the same question (present or absent), checked by the tests.
- Reproducibility: `python3 tests.py`, `python3 exp.py` and `python3 figs2.py` regenerate all evidence.

T8.5 Limitations of the measurements

Times are measured in an interpreted language on a shared machine with a single logical CPU, so absolute numbers are larger and slightly noisier than those of a compiled program. The very small times of Strategy C (below 1 µs at n = 1,000) are near the resolution of batch timing, so only their order of magnitude is meaningful. These limitations are addressed again in Task 10; they do not affect the comparison counts, which are exact and machine independent.

Task 9 – Record Experimental Performance (3 marks)

This task presents the measurements. All values come from one run of the experiment driver, and "search" results are averages over the same queries for all strategies.

T9.1 Number of key comparisons per search

Table 22. Average comparisons per search (successful, unsuccessful) and the worst count observed.

n	A succ.	A unsucc.	B succ.	B unsucc.	B worst	C succ.	C unsucc.	C worst
1,000	499	1,000	8.99	9.97	10	1.45	0.98	5
10,000	5,023	10,000	12.38	13.36	14	1.50	0.99	6
100,000	49,849	100,000	15.70	16.68	17	1.49	1.01	8
1,000,000	496,327	1,000,000	18.96	19.95	20	1.50	0.99	7

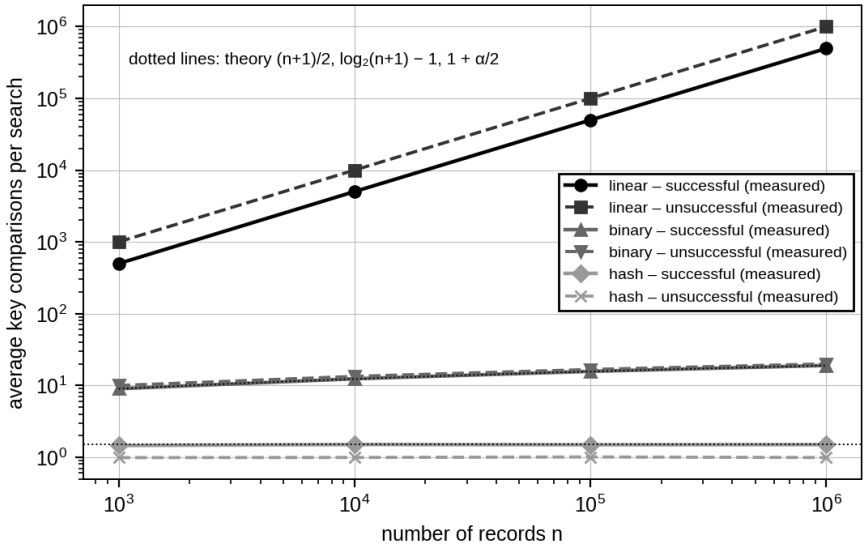


Figure 3. *Measured comparisons per search against n on logarithmic axes. The measured points lie on the dotted theoretical lines derived in Task 3.*

T9.2 Execution time per search

Table 23. Time per search in microseconds (µs). “Best” is the search for the first record (A) or the middle record (B).

n	A succ.	A unsucc.	A best	B succ.	B unsucc.	B best	C succ.	C unsucc.
1,000	17.5	36.9	0.318	1.855	1.765	0.194	0.160	0.156
10,000	185.5	389.4	0.241	2.359	2.458	0.476	0.356	0.203
100,000	2,423.7	4,563.6	0.260	4.736	4.767	0.254	0.895	0.769
1,000,000	23,751.7	45,792.6	0.256	7.618	6.391	0.318	1.035	0.887

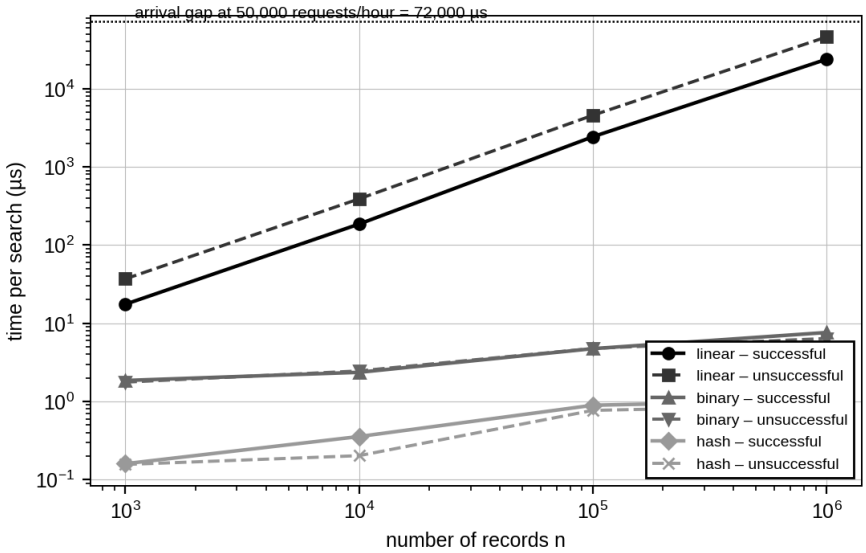


Figure 4. *Time per search against n (log–log). The dotted horizontal line is the 72 ms gap between consecutive requests at 50,000 requests per hour (Task 12).*

T9.3 Preprocessing cost

Table 24. Preprocessing measurements. built-in sorted() is shown only as a reference for a compiled sorting routine.

n	merge sort time	merge sort comparisons	built-in sorted()	hash build time	longest chain	load factor α
1,000	3.04 ms	8,677	156.96 µs	445.79 µs	5	0.9911
10,000	22.64 ms	120,388	1.44 ms	2.48 ms	6	0.9993
100,000	407.44 ms	1,536,095	35.32 ms	110.35 ms	8	1.0000
1,000,000	5.10 s	18,673,175	522.53 ms	917.14 ms	8	1.0000

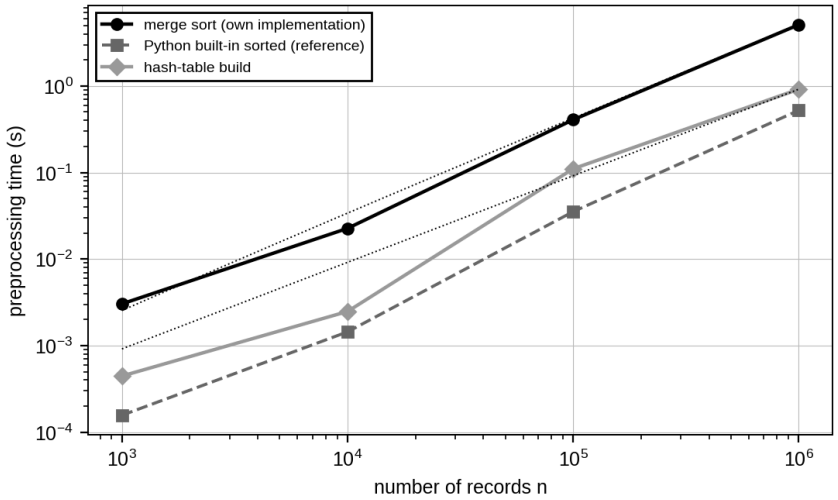


Figure 5. *Preprocessing time against n. The merge-sort curve is parallel to the dotted n log n line and the hash-build curve to the dotted linear line.*

T9.4 Memory used by the search structure

Table 25. Additional memory needed by Strategies B and C, measured with tracemalloc (the key objects themselves are shared and not counted).

Structure	n = 100,000	n = 1,000,000
Sorted array (references to the key objects)	1.2 MB	12.0 MB
Hash table (m chains and their lists)	8.4 MB	84.8 MB
Ratio hash ÷ sorted	7.0 ×	7.1 ×

T9.5 Distribution of chain lengths in the hash table

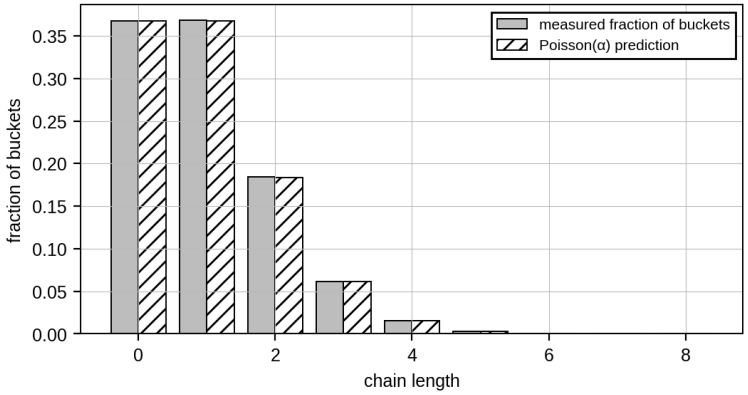


Figure 6. *Chain lengths in the hash table at n = 1,000,000 (α ≈ 1): measured fractions of buckets and the Poisson prediction.*

The measured distribution follows the Poisson prediction closely: 36.8 % of the buckets are empty, the longest chain has 8 records, and the mean chain length is α = 1.000. This confirms that the simple modulus hash spreads random registration numbers evenly.

Result of Task 9

Data for four sizes were recorded for comparisons, time per search (successful, unsuccessful, best case), preprocessing time and comparisons, memory and chain-length distribution, and presented in four tables and four graphs.

Task 10 – Compare Theory with Experimental Behaviour (4 marks)

The theoretical analysis of Tasks 3 to 7 predicted (i) the number of comparisons of every strategy, (ii) the growth of preprocessing and (iii) the order of growth of the running time. This task checks each prediction against the measurements of Task 9.

T10.1 Do the comparison counts agree with the derived formulas?

Table 26. Derived versus measured comparison counts.

Quantity	n	Theory	Measured	Difference
A successful: (n + 1)/2	1,000	500.5	498.6	-0.38 %
A successful: (n + 1)/2	1,000,000	500,000.5	496,326.6	-0.73 %
B successful: $\log_2(n + 1) - 1$	1,000	8.97	8.99	+0.24 %
B successful: $\log_2(n + 1) - 1$	1,000,000	18.93	18.96	+0.14 %
B unsuccessful: $\log_2(n + 1)$	1,000	9.97	9.97	+0.03 %
B unsuccessful: $\log_2(n + 1)$	1,000,000	19.93	19.95	+0.09 %
C successful: $1 + \alpha/2 - \alpha/2n$	1,000	1.495	1.455	-2.70 %
C successful: $1 + \alpha/2 - \alpha/2n$	1,000,000	1.500	1.497	-0.18 %
C unsuccessful: α	1,000	0.991	0.985	-0.66 %
C unsuccessful: α	1,000,000	1.000	0.988	-1.19 %
B sort: $n \log_2 n - 1.25n$	1,000	8,716	8,677	-0.44 %
B sort: $n \log_2 n - 1.25n$	100,000	1,535,964	1,536,095	+0.01 %
B sort: $n \log_2 n - 1.25n$	1,000,000	18,681,569	18,673,175	-0.04 %

Observation. All measured counts agree with the derived values to within about 1.3 %, with one exception: the successful hash search at n = 1,000 is 2.7 % below its expected value. Two causes explain the small deviations. First, a hash table built from 1,000 random keys is a single random layout, and its chain lengths can differ noticeably from the expectation taken over all layouts; the effect shrinks as n grows (−0.18 % at $n = 10^6$). Second, the averages come from 20,000 random queries, so they carry a sampling error; for the unsuccessful hash search at $n = 10^6$ the deviation of −1.2 % is under two standard errors. Strategy A's measured average of 496,327 comparisons at $n = 10^6$ differs from $(n + 1)/2 = 500{,}000.5$ by −0.7 %, which is also sampling error (the standard deviation of the mean is $n/\sqrt{(12 \cdot 20{,}000)} \approx 2{,}000$, so the difference is under two standard errors). The merge-sort count lies inside the derived bounds $\tfrac{1}{2} \cdot n \log n \le C \le n \log n - n + 1$ (for $n = 10^6$: 9.97 million ≤ 18.67 million ≤ 18.93 million) and agrees with the expected value $n \log n - 1.25n$ to within 0.5 %. The derivations of Task 3 are therefore confirmed.

T10.2 Does the running time follow the same growth?

Table 27. Measured time divided by the measured number of comparisons (successful searches).

n	A: ns per comparison	B: ns per comparison	C: ns per comparison
1,000	35.1	206.4	109.9
10,000	36.9	190.6	237.0
100,000	48.6	301.6	600.8
1,000,000	47.9	401.8	691.1

If time were exactly proportional to the comparison count, the three columns would be constant. For linear search the cost per comparison is almost constant, between 47.9 and 35.1 ns, so time grows as the number of comparisons: the log–log slope of the measured time between $n = 10^3$ and 10^6 is 1.04, close to the theoretical 1.00. For binary search the time per comparison is much higher (a recursive Python call costs several hundred nanoseconds) and increases with n; its time grows by a factor of 4.1 while the comparison count only doubles. For hashing the time per search grows by a factor of 6.5, although the comparison count is constant.

T10.3 Why do time and theory differ?

1. **Memory hierarchy.** The theory assumes that every comparison costs the same. In reality, a 1,000-record file fits in the processor cache, whereas 1,000,000 Python integers and the lists that reference them occupy tens of megabytes. Binary search touches about 20 widely separated memory locations, and hashing touches one bucket list and the integer object inside it: at $n = 10^6$ most of these accesses are cache misses, which cost 50–100 times as much as a cache hit. This explains why the time of B and C grows although their comparison counts hardly change. Linear search is not affected because it reads memory sequentially, which the hardware prefetches.
2. **Interpreter overhead.** In Python each function call, list access and integer comparison executes many machine instructions. This overhead is a roughly constant factor per comparison, but it differs between algorithms: the recursive binary search pays a function call per comparison (≈ 0.4 µs), whereas the linear loop pays only a loop iteration (≈ 50 ns). A compiled implementation would shrink these constants but not the growth rates.
3. **Hash computation and allocation.** Strategy C does extra work per search that is not a key comparison: computing K mod m and indexing the table. In the build phase it allocates one Python list per bucket (about 1,000,003 lists at $n = 10^6$), which is why the memory is 7 times larger than that of the sorted array and why the build time grows slightly faster than n.
4. **Merge sort implementation.** The own merge sort creates new lists by slicing at each level, copying every element about $\log_2 n$ times. The built-in sorted() is about ten times faster because it is compiled code and uses an adaptive algorithm (Timsort); both are $\Theta(n \log n)$ and the measured growth of the own merge sort follows n log n.
5. **Measurement noise.** Times of 0.2 – 1 µs per search are close to the resolution of the timer and are affected by scheduling and caching; they are averaged over 20,000 queries and three repetitions, but their last digit is not reliable.

T10.4 Which algorithm scales better?

Table 28. Measured growth of cost for a thousand-fold increase in n.

From n = 1,000 to 1,000,000	Growth of search time	Growth of preprocessing time
A linear	× 1,357	none
B binary	× 4.1	× 1,677
C hash	× 6.5	× 2,057

In terms of search time, Strategy A scales worst by far (a thousand-fold increase), Strategy B scales well, and Strategy C scales best (its growth is only the cache effect). In terms of preprocessing, the hash build grows roughly linearly and the sort grows by about two thousand times, as predicted by n log n. For a million records the two good strategies are separated by less than 10 µs per search, a difference that matters only at extreme request rates (Task 12).

T10.5 Measured break-even point for preprocessing

With 90 % successful searches at $n = 10^6$, one linear search takes 25.96 ms on average, one binary search 7.50 µs and one hash search 1.02 µs. Preprocessing pays for itself when $P \le q \cdot (t_A - t_x)$, so the break-even number of searches is $q^* = P/(t_A - t_x)$:

Table 29. Break-even number of searches (n = 1,000,000, 90 % successful) from the measurements.

Strategy	Preprocessing P	$t_A - t_x$	Break-even q*
B sort + binary search	5.10 s	25.95 ms	196 searches
C build + hash search	917.14 ms	25.95 ms	35 searches

Because the break-even values (196 and 35 searches) are tiny compared with the number of searches a university system receives, the preprocessing cost is justified quickly. At the full load of Task 12 (1.2 million searches per day) the sort is repaid in about 14 seconds of operation. The estimate from comparisons alone (about 37 searches, Task 3) is smaller than the measured value (196) because comparing in the interpreted merge sort costs more per comparison than a linear-scan comparison; this is a difference in constants, not in growth.

Result of Task 10

Comparison counts agree with the derived formulas to within about 1 %. Time per search follows the predicted growth for linear search (slope ≈ 1) and grows more slowly than any linear trend for B and C; the deviations from the ideal flat or logarithmic curves are explained by cache misses and interpreter overhead. Hashing scales best, binary search is a close second, linear search scales worst.

Task 11 – Recommend and Defend the Final Design (4 marks)

T11.1 Recommendation

Recommended design

Keep the registration numbers in a **sorted array** (merge sort as preprocessing) and search with **binary search** (Strategy B). Use sequential search only for files smaller than a few hundred records, where preprocessing is not worthwhile.

T11.2 Why Strategy B rather than A or C

Table 30. Evidence used for the recommendation (measured at n = 1,000,000).

Criterion	A Linear	B Binary (recommended)	C Hash table
Search comparisons, n = 10^6 (successful)	496,327	19.0	1.50
Worst case guaranteed?	Yes, n	**Yes, 20** ($\lfloor \log_2 n \rfloor + 1$)	No: n if keys collide; only probabilistic guarantee
Time per search, n = 10^6	23.75 ms	7.62 µs	1.03 µs
Preprocessing, n = 10^6	none	5.10 s (Θ(n log n))	917.14 ms (Θ(n))
Extra memory	none	12.0 MB	84.8 MB
Ordered output / range queries	no	**yes**	no
Update by daily batch	append	merge sorted batch, Θ(n + m)	insert batch, Θ(m)

- **Theoretical efficiency.** Binary search needs at most $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons for a million records, a Θ(log n) search with Θ(n log n) preprocessing (Tasks 3 and 7).
- **Experimental performance.** A successful search takes 7.62 µs and an unsuccessful one 6.39 µs against 23.75 ms and 45.79 ms for linear search, a speed-up of about 3,118 to 7,165 times.
- **Preprocessing.** Sorting costs 5.10 s for a million records and is repaid after about 196 searches (Task 10).
- **Why not the hash table?** It is faster per search, but the advantage is smaller than 10 µs, while it needs about 7 times as much memory, has no worst-case guarantee unless a randomised hash function is added, and cannot answer ordered or range queries. Hashing would be the better choice only if the lookups were the single most critical operation and memory were abundant.
- **Why not linear search?** It has no preprocessing and is the simplest, but it costs half a million comparisons per successful search at n = 10^6 and every unsuccessful search is a worst-case search.

T11.3 Trade-offs and limitations

- The sorted array must be re-ordered when records are added. A single insertion costs Θ(n) element moves, so records should be added in batches by merging (Task 12).
- The correctness of binary search depends on the sorted order; a bug in the sorting step corrupts every later search, which is why the test suite verifies the order and the answers.

- Binary search needs random access to the array; it is not suitable for data that is stored as a linked list or on a tape.
- The measured times belong to an interpreted implementation on one machine; the comparison counts and growth rates are machine independent.

T11.4 Does the design stay practical as the database grows?

Table 31. Projection to larger databases (extrapolated from the measurements with the n log n and log n laws; labelled as projections, not measurements).

n	search comparisons (worst)	merge-sort comparisons	projected sort time	projected search time
1,000,000	20	18,681,569	5.10 s	≈ 7.62 µs
10,000,000	24	220,034,967	59.46 s	≈ 9.14 µs
100,000,000	27	2,532,542,476	11.3 min	≈ 10.28 µs

A hundred-fold larger database (10^8 records) would need only 27 comparisons per search; the sort would take a few minutes and be done once a day. The weakness at that size is memory and caching rather than comparisons, for which disk-based structures such as B-trees would be considered. For the university in question, one million records, the sorted array remains practical.

Result of Task 11

Recommend binary search on a merge-sorted array: Θ(log n) search with a guaranteed worst case of 20 comparisons, Θ(n log n) one-off preprocessing repaid after a few hundred searches, small memory and ordered keys; hashing is the closest alternative, linear search is retained only for very small files.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt (8 marks)

Changed requirement. The system must now handle **50,000 search requests per hour** on a database of **1,000,000 records**, while new records are added **only once per day**. The original design of Task 11 (merge sort followed by binary search) is re-evaluated, and the decision is RETAIN, MODIFY or REDESIGN. Working assumptions: requests arrive evenly over 24 hours (so 50,000 × 24 = 1,200,000 requests per day), 90 % of requests are for existing students, the daily batch has 5,000 new records (0.5 % of the database), and one CPU core serves the requests.

T12.A Impact analysis (2 marks)

Table 32. What has changed.

	Original problem	Day-4 requirement
Database size	1,000 → 1,000,000+ records, growing	Fixed at 1,000,000 records between daily updates
Number of searches	Not specified; each design was compared per search and with q left open	q is known: 50,000/hour = 13.9 per second = one request every 72 ms = 1.2 million per day
Updates	Not specified	Once per day, in a batch
Performance goal	Efficiency and scalability	Sustained throughput: every request must be answered before the next one arrives
Preprocessing	Counted once, with unknown amortisation	Repeated daily and amortised over 1.2 million searches

Which assumptions are affected

- **Assumption of an unknown number of searches.** In Tasks 1–11 the number of searches q was left open and the break-even analysis (Task 10) had to be done for any q. Now q is known, so the total cost $P + q \cdot C$ can be evaluated for every design.
- **Assumption that preprocessing is a one-off.** It is now a recurring daily cost, which must be shown to be small compared with the total daily search effort.
- **Assumption of a growing database.** Within a day the database is static; it changes only at the daily update.

Why the frequency of searches is now important

The cost of a design over a day is Total = $P + q \cdot C$. With q = 1,200,000 the term $q \cdot C$ dominates: a method that costs 25.96 ms per search (linear search) consumes 8.7 hours of CPU time per day, whereas a method that costs 7.50 µs per search (binary search) consumes 9.0 seconds. The search frequency converts a difference of a few thousand-fold in cost per search into a difference of hours of computing time per day. It also fixes the deadline for each answer: a request must be served within $1/\lambda$ = 72 ms on average, otherwise requests queue up.

Why the low frequency of record updates influences the design

Because the database changes only once a day, a structure that is expensive to update but cheap to search is attractive: the update cost is paid once per day while the search benefit is collected 1.2 million times. A sorted array is such a structure: inserting one record into it would cost $\Theta(n)$ element moves, which would be unacceptable with continuous updates, but inserting a whole *batch* at once by merging costs only $\Theta(n + m)$. The same argument favours a hash table, which is also rebuilt or extended once a day. Frequent updates would have favoured balanced trees or hash tables with incremental insertion; infrequent updates make a static structure with a daily batch update the simplest and fastest choice.

T12.B Evaluation of the original design (2 marks)

Expected behaviour under 50,000 searches per hour

Table 33. Behaviour of the three designs at 50,000 requests per hour (90 % successful searches; times measured at n = 1,000,000).

Design	Service time per request	Utilisation of one core ρ = λ·t	CPU per day for searches	Prep. per day	Maximum sustainable rate (single core)
A Linear	25.96 ms	36.0497 %	8.65 h	none	39 req/s = 0.1 million/h
B Binary (original: full daily re-sort)	7.50 µs	0.0104 %	8.99 s	5.10 s	133,420 req/s = 480.3 million/h
C Hash (daily rebuild)	1.02 µs	0.0014 %	1.22 s	917.14 ms	980,427 req/s = 3,529.5 million/h

The original design (B) copes easily: the average request takes 7.50 µs, so one core is busy 0.0104 % of the time at a load of 13.9 requests per second, and the maximum sustainable rate is about 133,420 requests per second, roughly 9,606 times the required load. Strategy C is faster still (1.02 µs), and A is the only design whose behaviour is a concern: its service time is 25.96 ms, so ρ = 0.36 and the core is 36 % busy. The queueing delay of an M/M/1 system (an assumption of random arrivals) would make the mean response time $s/(1 - \rho)$ = 40.59 ms for A, against 7.50 µs for B. Moreover A can sustain only 38.5 requests per second, which is 2.8 times the required load: a threefold burst would overload it.

Is the preprocessing cost acceptable?

The daily full re-sort costs 5.27 s for 1,000,000 + 5,000 records, i.e. 0.0061 % of the day, which is acceptable in absolute terms. It is nevertheless *wasteful*: the sort repeats the ordering work of a database that is already sorted, apart from the 5,000 new records. The cost of re-sorting is $\Theta(n \log n)$, whereas only $\Theta(n + m)$ is needed to place m new records into an already sorted file.

Could another approach give better overall performance?

The hash table would reduce the search time from 7.50 µs to 1.02 µs and the daily preprocessing from 5.10 s to 917.14 ms (or to 4.64 ms if the 5,000 new records are inserted into the existing table). Over a day the total CPU time would fall from 14.09 s to 2.14 s. The saving of roughly 11.9 seconds a day is real, but it is a saving on a quantity that is already about 0.02 % of the machine; it does not remove a bottleneck, and it would cost 7 times as much memory and the ordered access to the registration numbers.

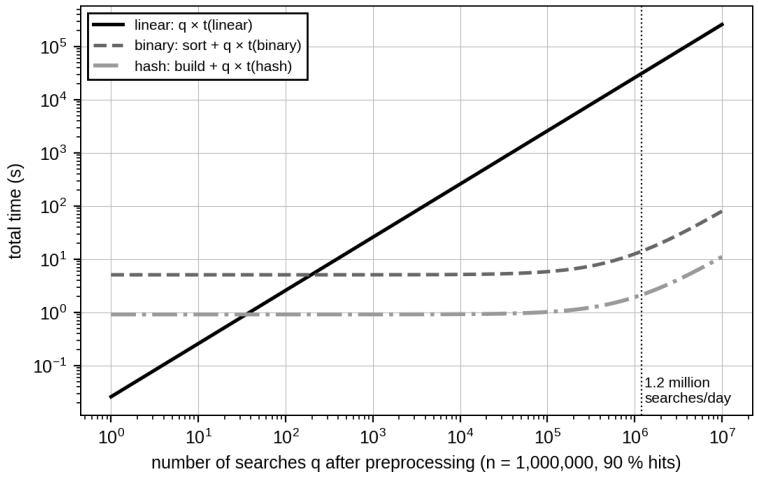


Figure 7. *Total time P + q·t for the three designs against the number of searches q (n = 1,000,000, 90 % successful). Both preprocessed designs are cheaper than linear search after a few hundred searches; the dotted line marks the 1.2 million searches of one day.*

T12.C Adaptation or redesign (2 marks)

Decision: MODIFY

Retain binary search on a sorted array as the search strategy, and **modify the daily preprocessing**: instead of re-sorting all 1,000,000 + m records every night, sort only the m new records and **merge** them into the existing sorted array in linear time.

Revised algorithm

```
DailyUpdate(Sorted[0..n-1], NewBatch[0..m-1])
  B <- MergeSort(NewBatch)                  // m log m comparisons
  C <- empty array of size n + m
  i <- 0 ; j <- 0 ; k <- 0
  WHILE i < n AND j < m DO                  // linear merge: at most n + m - 1 comparisons
    IF Sorted[i] <= B[j] THEN C[k] <- Sorted[i] ; i <- i + 1
    ELSE C[k] <- B[j] ; j <- j + 1
    k <- k + 1
  copy the rest of Sorted or of B into C
  publish C as the new Sorted array         // swap in C only when complete
```

Pseudocode of the revised daily update (Strategy B, modified).

What has changed and its effect on complexity

Table 34. Effect of the modification.

	Original update (full re-sort)	Modified update (sort batch + merge)
Algorithm	MergeSort of all n + m records	MergeSort of m records, then linear Merge
Comparisons (theory)	$(n + m)\log_2(n + m) - 1.25(n + m) \approx 18.7$ million	$m \log_2 m - 1.25m + (n + m - 1) \approx 1.05$ million
Comparisons (measured)	18,673,175	1,004,968 (merge step)
Time (measured)	5.27 s	424.52 ms
Complexity	$\Theta((n + m) \log (n + m))$	$\Theta(m \log m + n + m)$
Search after the update	$\Theta(\log n)$, unchanged	$\Theta(\log n)$, unchanged
Memory during the update	a second copy of the array	a second copy of the array (searches continue on the old copy)

The measured saving is 12.4 times in time and 19 times in comparisons. The merged array was verified to be identical to the array obtained by sorting everything again, and the new records were found by binary search after the update. The search algorithm and its worst case of 20 comparisons are untouched.

Effect of the batch size

Table 35. Comparison counts of the two update methods for different daily batch sizes (n = 1,000,000).

Records added per day m	batch sort ≈ m log m − 1.25m	merge ≈ n + m − 1	total (modified)	full re-sort ≈ (n+m) log(n+m) − 1.25(n+m)	gain
1,000	8,716	1,000,999	1,009,715	18,701,694	18.5 ×
5,000	55,189	1,004,999	1,060,188	18,782,208	17.7 ×
50,000	717,982	1,049,999	1,767,981	19,689,556	11.1 ×
500,000	8,840,784	1,499,999	10,340,783	28,899,797	2.8 ×

The gain decreases as the batch becomes larger, because the merge always costs n + m comparisons; but for a daily batch of up to tens of thousands of records the modification is at least an order of magnitude cheaper.

Why not redesign?

A redesign (for example to a hash table) would change the search algorithm. It would not be justified here: the search is already 9,606 times faster than required, so a faster search brings no benefit, while the hash table costs 7 times as much memory, loses ordered access and needs a randomised hash to avoid its worst case. Redesign would be revisited if the request rate grew by several orders of magnitude (above about 10^5 requests per second on one core) or if only exact-match lookups were ever required and memory were not limited. Linear search (the “retain A” option) is rejected because it uses 36 % of a core and would saturate under a threefold burst.

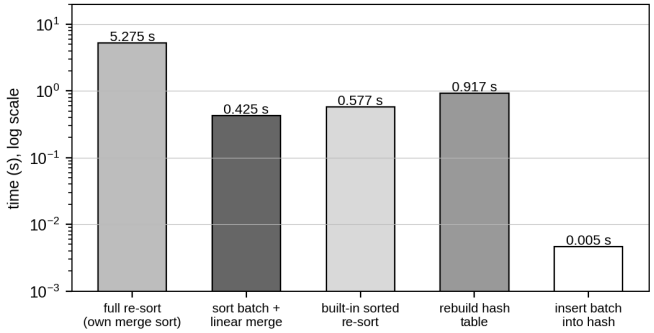


Figure 8. *Daily update strategies for a batch of 5,000 new records in a database of 1,000,000 (measured, log scale).*

T12.D Evidence-based defence (2 marks)

Table 36. Evidence supporting the final design.

Evidence	Finding	Implication for the decision
Mathematical complexity	Search Θ(log n) ≤ 20 comparisons; full update Θ(n log n); batch update Θ(m log m + n + m)	Search is already negligible; only the update can be improved, and it can be improved by a large factor
Search frequency	13.9 requests/s; binary search uses 0.0104 % of one core; linear search 36 %	Linear search is rejected; B and C are both far below capacity
Preprocessing cost	Full re-sort 5.27 s; batch merge 424.52 ms; hash rebuild 917.14 ms	Merging removes 92 % of the daily preprocessing without changing the search
Dataset size	n = 10^6, fixed daily; m = 5,000 new records (0.5 %)	The database is static between updates, so a static sorted array is ideal; since m n a merge is nearly optimal
Break-even	Sorting repaid after about 196 searches, i.e. within 14 seconds of operation	The preprocessing is justified many times over
Trade-offs	Hash: faster by a few µs but 7× memory, no order, collision risk; sorted array: ordered, 12 MB, guaranteed 20 comparisons	Extra speed of hashing has no value at 13.9 requests/s; the other properties of B do

Logical chain of the decision. (1) At 13.9 requests per second the 72 ms gap between requests is enormous compared with the 8 µs of a binary search (a margin of 9,606×), so the search algorithm does not need to be replaced. (2) Linear search would work, but it would occupy 36 % of a core and has almost no headroom, so it must not be retained. (3) The only inefficiency in the original design is the nightly re-sort of an almost sorted file, which can be replaced by batch merging at a cost of Θ(n + m) with a measured speed-up of 12 times. (4) The daily update must publish the new array only when complete, so that searches never see a half-built array. The final design is therefore: **binary search retained, preprocessing modified**.

Sensitivity. If the request rate rose ten-fold to 500,000 per hour, binary search would still use only 0.1 % of a core. If the daily batch grew to 500,000 records (half of the database), the gain of merging would fall to under 3× (see the table above) and shrink further as the batch approaches the size of the database. The decision should be revisited only if the request rate or the update frequency changes by orders of magnitude.

Result of Task 12

MODIFY: retain binary search on a sorted array (Θ(log n), ≤ 20 comparisons, ≈ 8 µs per search, 0.01 % of one core at 50,000 requests per hour) and replace the nightly full re-sort (Θ(n log n), 5.3 s) by sorting the daily batch and merging (Θ(n + m), 0.42 s).

Conclusion

This report designed three distinct strategies for retrieving a student record by registration number: a brute-force sequential search (iterative), a recursive binary search on a merge-sorted file (decrease and conquer), and a hash table with chaining (space–time trade-off). The basic operation of each was identified, the exact cost functions were derived, and the best, average and worst cases were explained. The recursive search satisfies $T(n) = T(\lfloor n/2 \rfloor) + 1$, whose solution is $\lfloor \log_2 n \rfloor + 1$, and the merge-sort preprocessing satisfies $W(n) = 2W(n/2) + n - 1$, whose solution is $n \log_2 n - n + 1$.

Experiments with 1,000 to 1,000,000 records confirmed the analysis: the measured comparison counts agree with the derived formulas to within about one per cent, and the time per search for a million records is 23.75 ms for linear search, 7.62 µs for binary search and 1.03 µs for hashing. Differences between the growth of time and the theory were explained by the memory hierarchy and by interpreter overhead.

The recommended original design is binary search on a sorted array. Under the Day-4 requirement of 50,000 searches per hour with daily updates the decision is to **modify** the design: the search remains unchanged, and the nightly full re-sort is replaced by sorting the small daily batch and merging it into the sorted array, which reduces the update cost from Θ(n log n) to Θ(n + m log m).

Appendix A – Source Code (key parts)

The listings below are taken from `algos.py`, the module used for all experiments. The complete files `algos.py`, `tests.py` and `exp.py` regenerate every table and graph of this report.

```
# ---------------- Strategy 1: sequential (brute-force) search, iterative -------------
def linear_search(a, key):
    for i in range(len(a)):
        if a[i] == key:
            return i
    return -1

def linear_search_counted(a, key):
    comps = 0
    for i in range(len(a)):
        comps += 1
        if a[i] == key:
            return i, comps
    return -1, comps
```

Listing A.1 – Strategy A: sequential search (plain and counted variants).

```
# ---------------- Strategy 2: binary search (decrease and conquer), recursive --------
def binary_search(a, lo, hi, key):
    if lo > hi:
        return -1
    mid = (lo + hi) // 2
    if a[mid] == key:
        return mid
    if key < a[mid]:
        return binary_search(a, lo, mid - 1, key)
    return binary_search(a, mid + 1, hi, key)

def binary_search_counted(a, lo, hi, key, c=0):
    if lo > hi:
        return -1, c
    mid = (lo + hi) // 2
    c += 1                       # one three-way comparison of key with a[mid]
    if a[mid] == key:
        return mid, c
    if key < a[mid]:
        return binary_search_counted(a, lo, mid - 1, key, c)
    return binary_search_counted(a, mid + 1, hi, key, c)

# preprocessing for strategy 2: merge sort (divide and conquer)
```

Listing A.2 – Strategy B: recursive binary search (plain and counted variants).

```
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    L, R = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]:
            out.append(L[i]); i += 1
        else:
            out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out

CNT = [0]
```

Listing A.3 – Strategy B preprocessing: merge sort.

```
def merge_sort_counted(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    L, R = merge_sort_counted(a[:mid]), merge_sort_counted(a[mid:])
    out, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        CNT[0] += 1
        if L[i] <= R[j]:
            out.append(L[i]); i += 1
        else:
            out.append(R[j]); j += 1
    out.extend(L[i:]); out.extend(R[j:])
    return out

def merge_two(A, B, counter=None):
    """linear merge of two sorted lists (used for the daily update in Task 12)"""
    out, i, j = [], 0, 0
    while i < len(A) and j < len(B):
        if counter is not None: counter[0] += 1
        if A[i] <= B[j]:
            out.append(A[i]); i += 1
        else:
            out.append(B[j]); j += 1
    out.extend(A[i:]); out.extend(B[j:])
    return out
```

Listing A.4 – Counted merge sort and the linear merge of two sorted lists (daily update of Task 12).

```
# ---------------- Strategy 3: hashing with chaining (space-time trade-off) -----------
def next_prime(x):
    def isp(v):
        if v < 2: return False
        if v % 2 == 0: return v == 2
        d = 3
        while d * d <= v:
            if v % d == 0: return False
            d += 2
        return True
    while not isp(x):
        x += 1
    return x
```

Listing A.5 – Choosing the prime table size m.

```
class HashIndex:
    def __init__(self, m):
        self.m = m
        self.b = [[] for _ in range(m)]
        self.size = 0
    def insert(self, key):
        self.b[key % self.m].append(key); self.size += 1
    def build(self, keys):
        for k in keys:
            self.insert(k)
    def search(self, key):
        for x in self.b[key % self.m]:
            if x == key:
                return True
        return False
    def search_counted(self, key):
        c = 0
        for x in self.b[key % self.m]:
            c += 1
            if x == key:
                return True, c
        return False, c
```

Listing A.6 – Strategy C: hash table with chaining (build and search).

```
class UniversalHashIndex(HashIndex):
    """randomised hash h(k) = ((a*k + b) mod P) mod m  (mitigation for adversarial keys)"""
    P = (1 << 61) - 1
    def __init__(self, m, seed=1):
        super().__init__(m)
        r = random.Random(seed)
        self.a = r.randrange(1, self.P); self.c = r.randrange(0, self.P)
    def h(self, k): return ((self.a * k + self.c) % self.P) % self.m
    def insert(self, key):
        self.b[self.h(key)].append(key); self.size += 1
```

Listing A.7 – Universal hash index used to remove the adversarial worst case.

Appendix B – Raw Measurements

The table lists the main raw values of the experiment (results.json) used in this report. Times are in microseconds per search unless stated.

Table 37. Raw experimental values.

Quantity	n = 1,000	n = 10,000	n = 100,000	n = 1,000,000
Merge sort time (s)	0.003039	0.022642	0.407441	5.096832
Merge sort comparisons	8,677	120,388	1,536,095	18,673,175
Built-in sorted() time (s)	0.000157	0.001443	0.035320	0.522534
Hash build time (s)	0.000446	0.002482	0.110349	0.917142
Longest chain	5	6	8	8
A successful comparisons	498.6	5,022.8	49,848.5	496,326.6
A successful time (µs)	17.501	185.506	2,423.732	23,751.673
A unsuccessful time (µs)	36.922	389.436	4,563.628	45,792.649
A worst-case time (µs)	38.474	387.685	5,036.502	43,434.061
B successful comparisons	8.988	12.378	15.704	18.959
B unsuccessful comparisons	9.970	13.364	16.681	19.950
B worst comparisons	10	14	17	20
B successful time (µs)	1.855	2.359	4.736	7.618
B unsuccessful time (µs)	1.765	2.458	4.767	6.391
C successful comparisons	1.455	1.502	1.489	1.497
C unsuccessful comparisons	0.985	0.991	1.006	0.988
C worst comparisons	5	6	8	7
C successful time (µs)	0.160	0.356	0.895	1.035
C unsuccessful time (µs)	0.156	0.203	0.769	0.887

Appendix C – References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022 – hash tables, merge sort, recurrences and the Master Theorem.
2. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012 – basic operations, mathematical analysis of recursive algorithms, brute force and decrease-and-conquer.
3. R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011 – binary search, hashing and empirical analysis of running time.
4. D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed., Addison-Wesley, 1998 – average cost of merge sort, sequential and binary search, hashing.
5. J. L. Carter and M. N. Wegman, “Universal classes of hash functions”, *Journal of Computer and System Sciences*, 18(2), 1979 – universal hashing.